

INSO & MAIS



# Procesamiento de datos

Miguel Ángel Fernández Díaz  
2024-2025



# Tema 4:

## Spark SQL

1. Introducción a SparkSQL
2. APIs de Spark
3. SparkSession
4. Conectores

# 1. Introducción a SparkSQL

Módulo de Spark que proporciona una abstracción de tabla proporcionando un esquema

Acerca las capacidades de un framework de Big Data a herramientas de Business Intelligence

Acceso uniforme a datos

Conecta a cualquier fuente de datos de la misma manera

Mezcla datos entre diferentes fuentes de datos

Hereda la tolerancia a fallos y la computación distribuida de los RDDs y del Engine Core y las optimizaciones del Dataset

# Spark SQL: ¿Por qué surge?

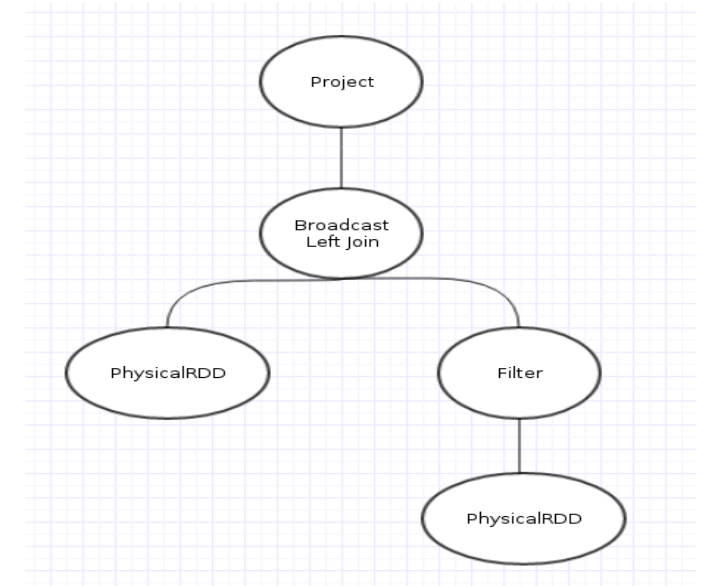
El estándar SQL se impone ante el lenguaje NoSQL por su fuerte cohesión con las herramientas de BI

La API de RDDs no tiene concepto de esquema

Los RDDs no aprovechan información sobre el contexto de ejecución:

- Optimiza la ejecución de las sentencias a través árboles de ejecución, cachés, estadísticas...

Hive tenía carencias en cuanto a rendimiento y la tolerancia a fallos



Acceso unificado a una gran variedad de fuente de datos gracias a la API de Datasources

Escalabilidad (RDDs)

Tolerancia a fallos (RDDs)

Motor de ejecución optimizado

Integración con RDDs, DataFrames y Datasets

Compatibilidad con sentencias Hive

Compatibilidad con Hive metastore

Extensibilidad

Push-down de filtros

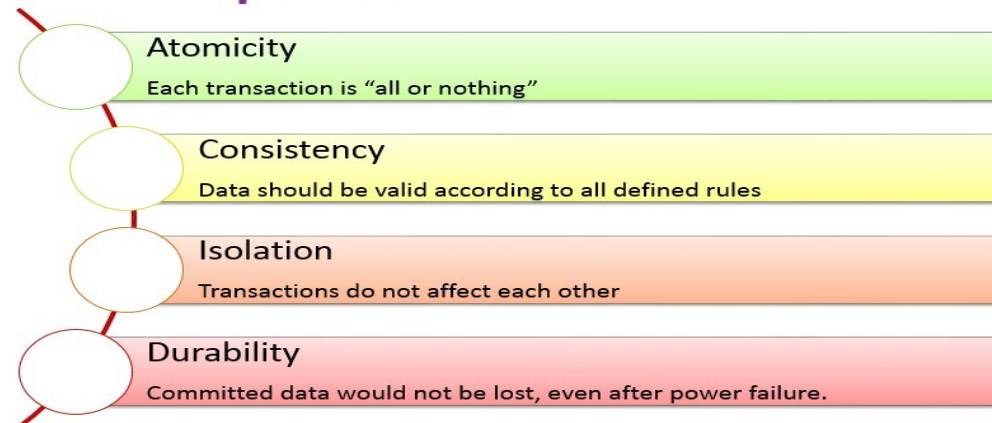
Dado que accede a fuentes de datos muy heterogéneas:

- No hay concepto de Primary Key
- No hay concepto de Foreign Key
- No hay concepto de índices
- No hay concepto de triggers
- No hay concepto de transacciones ACID
- No hay concepto de roles
- No hay concepto de esquema
- No hay lenguaje procedural

2 tipos de tablas

- Tabla externa
- Table manejada

## ACID Properties





# Spark SQL: ¿Qué no es?

NO es un data warehouse

NO es una base de datos relacional

NO es una herramienta transaccional

NO es una herramienta comercial

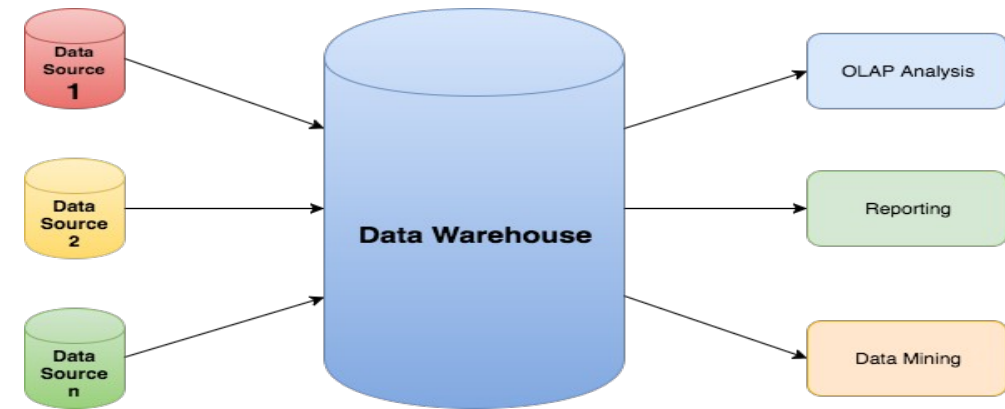
NO es una herramienta gráfica

NO es una herramienta para sentencias online

NO es una herramienta que solo utilice memoria

NO es una cola para el envío de datos

NO es solo una interfaz SQL



# Spark SQL: Casos de uso

Conectar herramientas de BI con un data lake

Realizar analítica avanzada en Batch

Realizar analítica avanzada en Streaming

- Agregaciones en tiempo real (cubos OLAP)

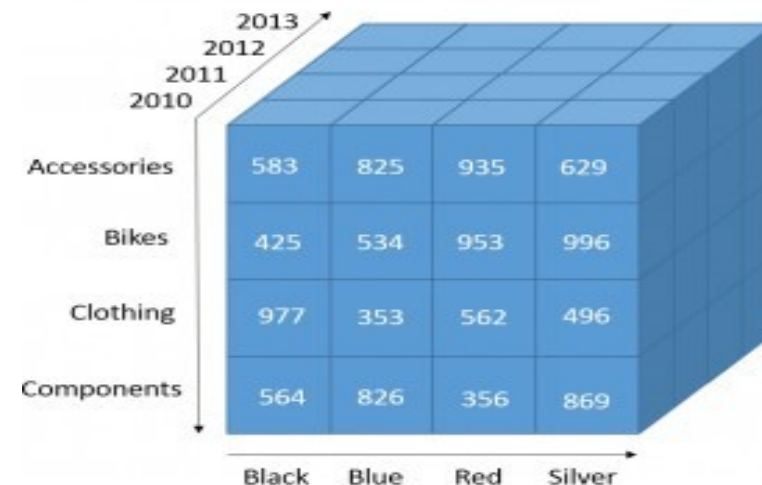
Machine learning a través de DataFrames

Proporcionar esquema a datos desestructurados

ETL

Ingesta de datos

Analítica avanzada en la nube



## 2. APIs de Spark

## RDD

Nivel más bajo, procesamiento funcional de datos  
desestructurados (Spark 0.1+)

## Dataframe

Procesamiento relacional de datos estructurados (Spark 1.3+)

## Dataset

Combine tanto procesamiento funcional como relacional  
(Spark 1.6+)

Un dataset es una colección **fuertemente tipada** que representa objetos de un dominio específico. Ejemplo, una colección de facturas

Esta colección puede ser transformada en paralelo usando **operaciones funcionales** (map, filter) **y relacionales** (select, where)

Proporcionan **seguridad a nivel de compilación**, lo que significa que reduce la probabilidad de errores antes de desplegar la aplicación

Contienen **esquema** (nombre de propiedad y tipo)

Utiliza un encoder para poder realizar una conversión directa entre objetos de la JVM y una representación de tabla en formato binario, lo cual permite una mejor utilización de la memoria

Existen Encoders de **tipos primitivos de Scala y case class**

Optimizado gracias al motor de SparkSQL

2 tipos de operaciones: **transformaciones y acciones**

Un Dataset es una colección fuertemente tipada de objetos de dominio específico que puede ser transformado en paralelo usando operaciones funcionales o relacionales. Cada Dataset también **tiene una vista sin tipado llamada DataFrame**, que es un Dataset de filas.

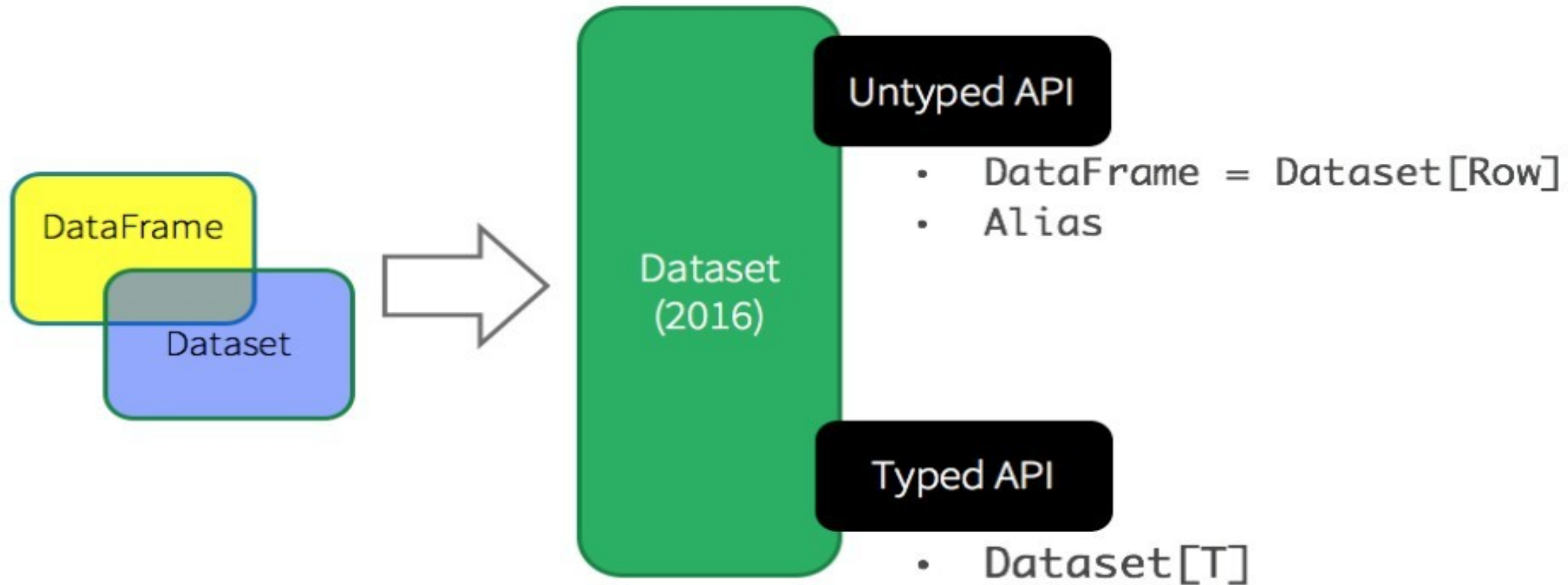
Tiene un DSL a medida y se ejecuta sobre una gestión a medida de memoria

Proporciona un DSL muy similar al de un RDD

**Combina** lo mejor de las **APIs de RDD y de Dataframe**

La mayoría de las operaciones de un Dataset se ejecutan sobre código y serialización a medida

# Unified Apache Spark 2.0 API



# ¿Cómo se crea un Dataset?

```
{“ip”: “127.0.0.1”, “name”: “droneA”, “id”: 25, “timestamp”: 1486596321,  
“speed”: 12}
```

```
xx case class DroneData(ip: String, name: String, id: Long, timestamp: Long,  
speed: Long)
```

```
xx val ds = spark.read.json(“droneFlight.json”).as[DroneData]
```

1. Spark lee el JSON, infiere el esquema y genera una colección de DataFrames.
2. Spark convierte los datos en DataFrame = Dataset[Row], una colección de objetos genéricos de tipo Row, ya que no conoce el tipo exactamente.
3. Spark convierte el Dataset[Row] —> Dataset[DroneData] con tipos específicos de Scala como objetos de la JVM



```
import org.apache.spark.sql._  
  
{“ip”: “127.0.0.1”, “name”: “droneA”, “id”: 25, “timestamp”:  
1486596321, “speed”: 12}  
  
xx case class DroneData(ip: String, name: String, id: Long,  
timestamp: Long, speed: Long)  
  
xx val ds = spark.read.json(“droneFlight.json”).as[DroneData]  
  
xx val dsAvgSpeed = ds.filter(d => {d.speed > 10}).map(d =>  
(d.ip, d.speed)).groupBy($"_2").avg()  
  
xx dsAvgSpeed.show(500, false)
```

Un DataFrame es un **Dataset** organizado en **columnas** nombradas.

Conceptualmente **equivalente a tabla de una base de datos** relacional, pero con grandes optimizaciones gracias al motor de Spark SQL

Se puede construir a partir de ficheros estructurados, tablas de Hive, tablas de bases de datos externas o RDDs existentes.

API para Scala, Java, Python y R

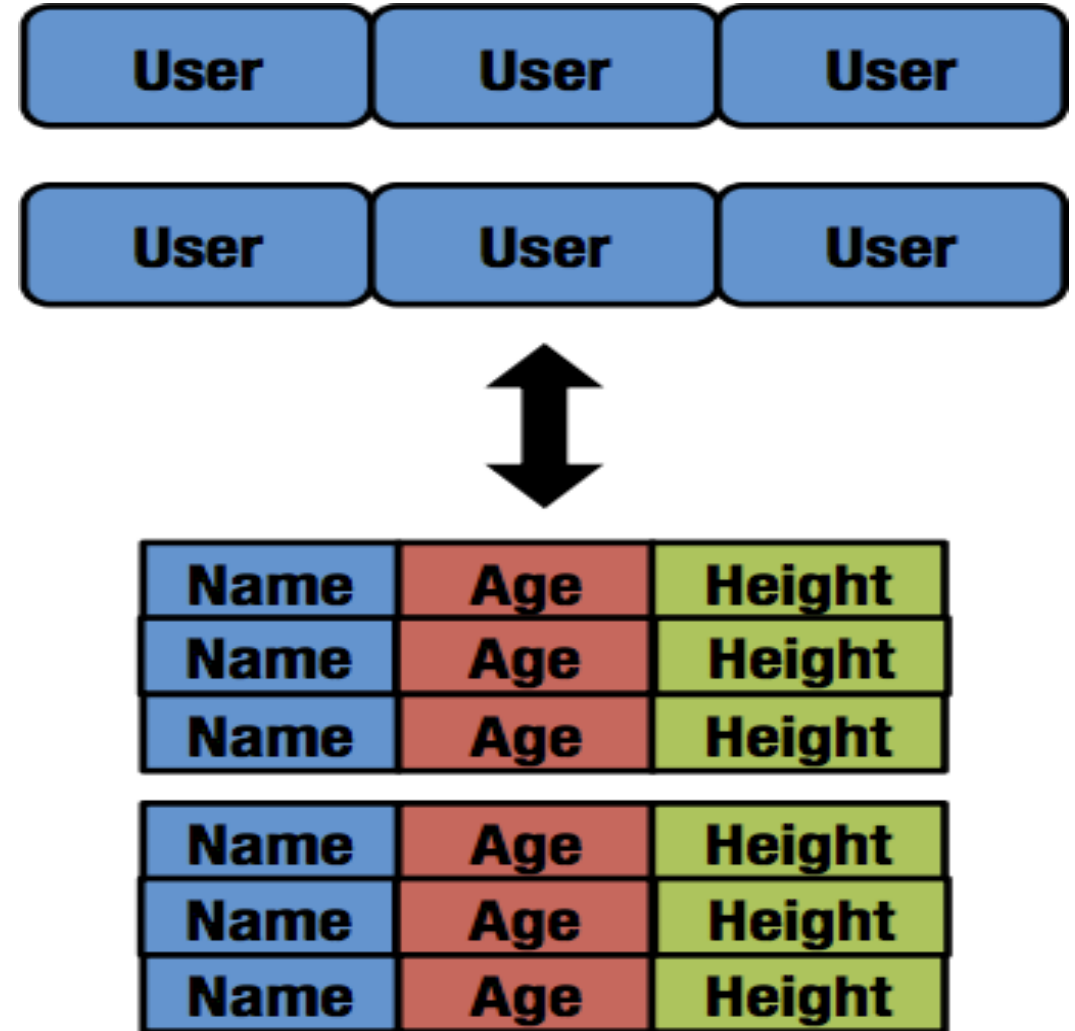
Internamente es un **Dataset[Row]**

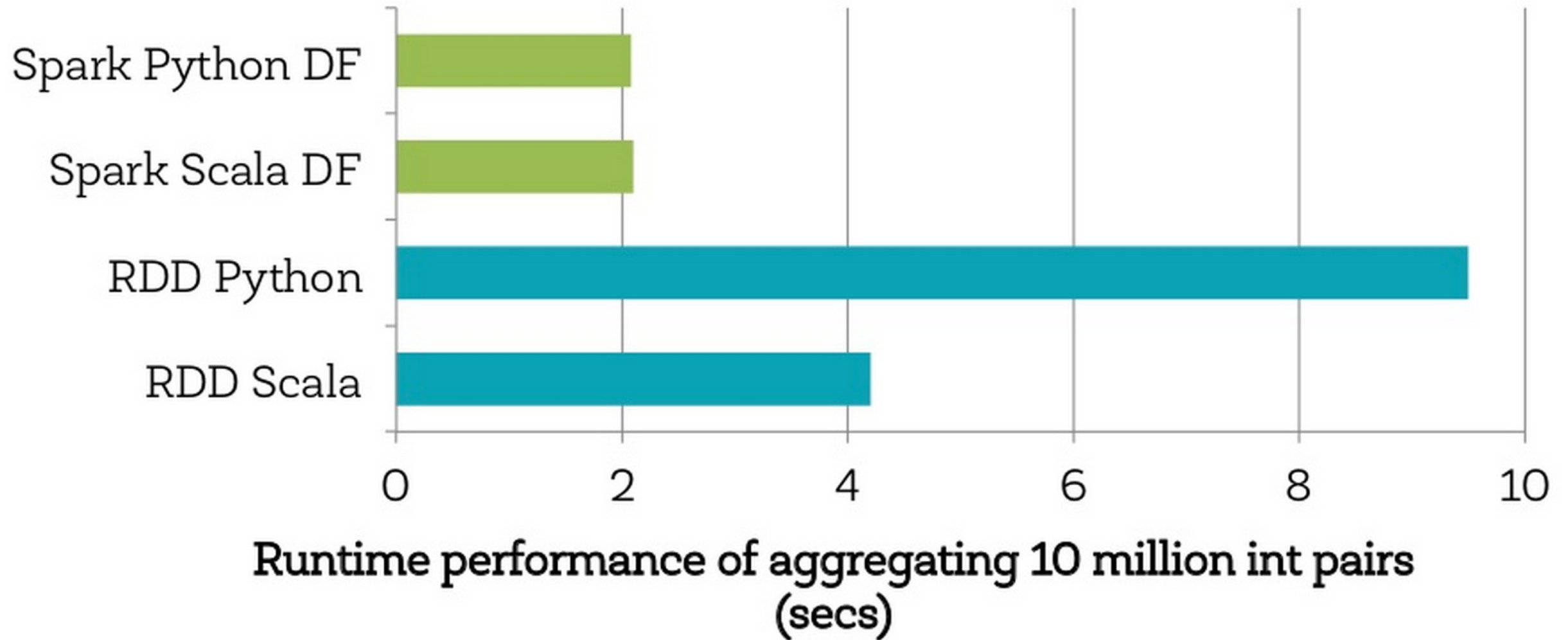
2 tipos de operaciones: **transformaciones y acciones**

SPARK + RDD Transformamos  
objecto no estructurados de forma  
opaca al modelo de datos

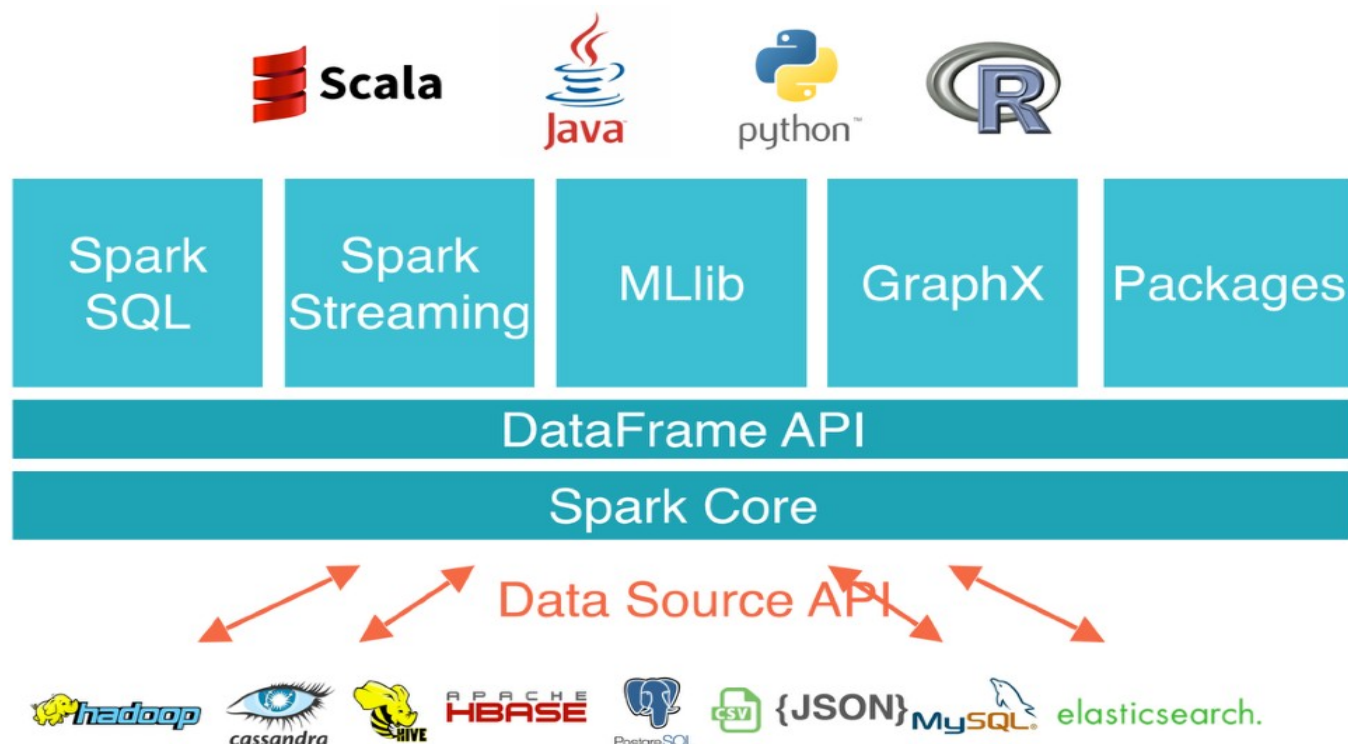
SQL + DataFrame

Se infiere un schema o un tipo de  
objeto y podemos trabajar con el  
como si fuera SQL

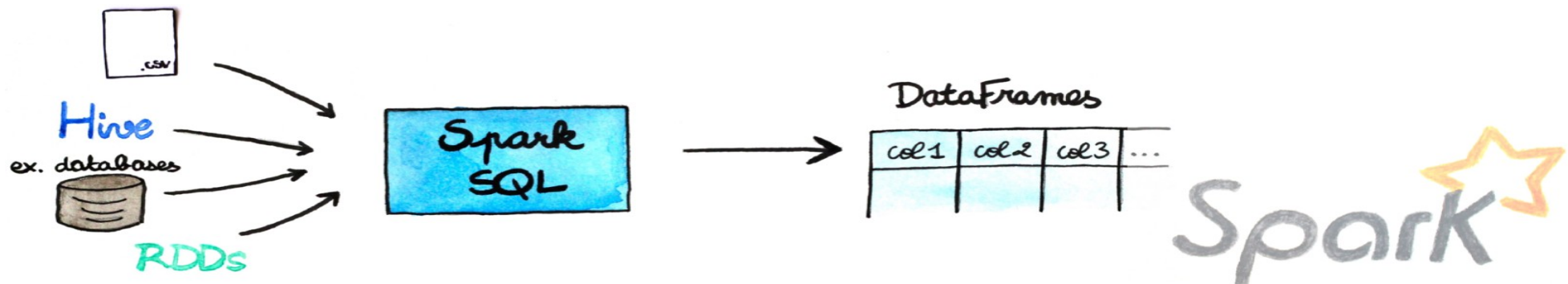


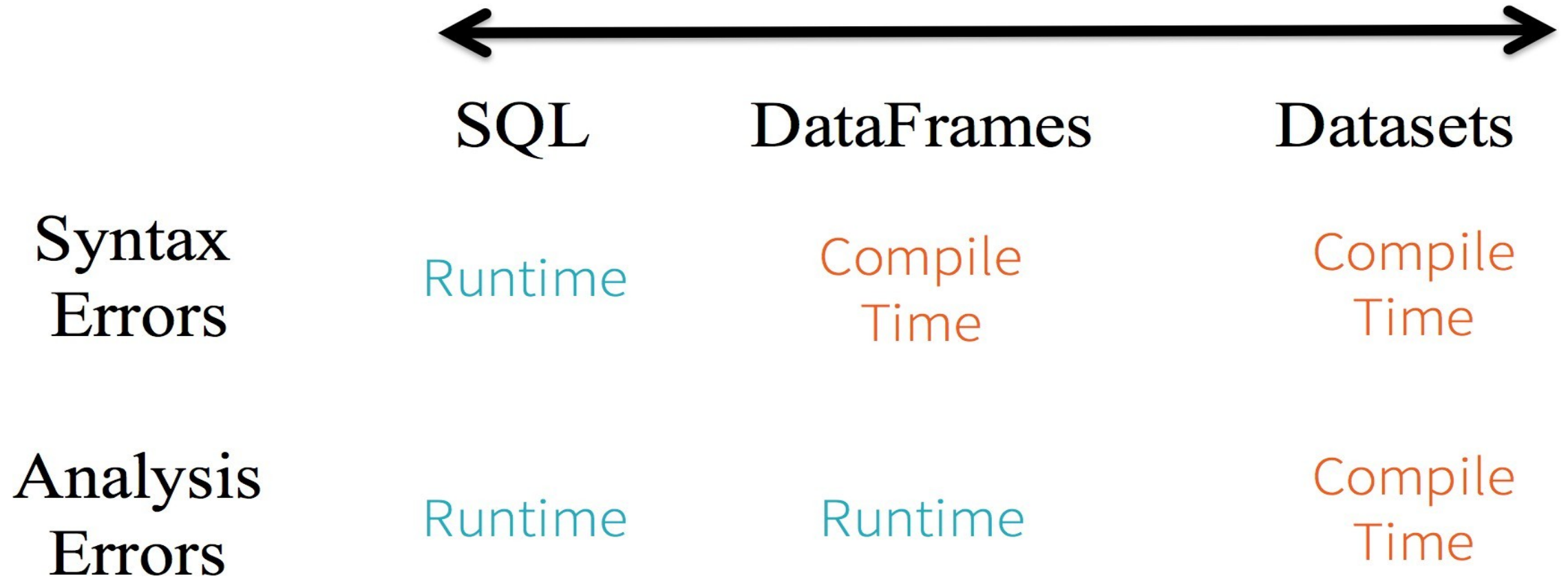


Un DataFrame es una colección distribuida de datos organizados en **columnas nombradas**. Conceptualmente, es equivalente a una tabla en una base de datos relacional o a un marco de datos en R/Python, pero con optimizaciones más avanzadas en su funcionamiento interno. Los DataFrames pueden construirse a partir de una **amplia variedad de fuentes**, como archivos de datos estructurados, tablas en Hive, bases de datos externas o RDDs existentes.



```
import org.apache.spark.sql._  
val sparkSession = SparkSession.builder().master("local[*]").getOrCreate()  
import sparkSession.implicits._  
  
case class Person(name: String, age: Int)  
  
val people =  
  sc.textFile("examples/src/main/resources/people.txt").map(_.split(",")).map(p  
=> Person(p(0), p(1).trim.toInt)).toDF()  
  
people.registerTempTable("people")  
  
val teenagers = sparkSession.sql("SELECT name, age FROM people WHERE age >= 13 AND age  
<= 19")
```





//User defined function

```
xx val dataset = Seq((0, "hello"),(1,"world")).toDF("id","text")
```

```
xx import org.apache.spark.sql.functions._
```

```
xx val upperUDF = udf((text: String) => text.toUpperCase)
```

```
xx dataset.withColumn("upper", upperUDF($"text")).show
```

```
xx dataset.withColumn("upper", upperUDF(col("text"))).show
```

```
xx spark.udf.register("myUpper", upperUDF)
```

```
xx spark.catalog.listFunctions.filter('name like "%upper%").show(false)
```

```
xx spark.sql("SELECT myUpper('prueba)').show
```



User defined aggregate function

- Permite realizar funciones de agregación teniendo en cuenta todos los valores de una columna
- class GroupConcat extends UserDefinedAggregateFunction

➔Initialize: estado inicial del acumulador

➔Update: por cada fila, actualizar acumulador

➔Merge: combinar resultados parciales de particiones (shuffle)

➔Evaluate: calcular y devolver resultado

- spark.udf.register("group\_concat", new GroupConcat)
- spark.sql("select group\_concat(role) from example")
- Resultado: arquitecto, ingeniero, manager, programador

```
class GroupConcatUDAF extends UserDefinedAggregateFunction {
  def inputSchema: StructType = StructType(StructField("value", StringType) :: Nil)
  def bufferSchema: StructType = StructType(StructField("concatValues", StringType) :: Nil)
  def dataType: DataType = StringType
  def deterministic: Boolean = true
  def initialize(buffer: MutableAggregationBuffer): Unit = { buffer(0) = "" }
  def update(buffer: MutableAggregationBuffer, input: Row): Unit = {
    val currentValue = buffer.getString(0)
    val newValue = if (currentValue.isEmpty) input.getString(0) else s"$currentValue, ${input.getString(0)}"
    buffer(0) = newValue
  }
  def merge(buffer1: MutableAggregationBuffer, buffer2: MutableAggregationBuffer): Unit = {
    val currentValue1 = buffer1.getString(0)
    val currentValue2 = buffer2.getString(0)
    val newValue = if (currentValue1.isEmpty) currentValue2 else if (currentValue2.isEmpty) currentValue1 else
    s"$currentValue1, $currentValue2"
    buffer1(0) = newValue
  }
  def evaluate(buffer: MutableAggregationBuffer): Any = {
    buffer.getString(0)
  }
}

spark.udf.register("group_concat", new GroupConcatUDAF)
```

# 3. SparkSession

## SparkContext

- Es la aplicación Scala que provoca la formación del Spark Cluster. Engloba el Spark Driver y la comunicación con sus executors. Permite el acceso a la API de RDDs

## SQLContext

- Es el punto de entrada a SparkSQL y es una capa por encima del SparkContext. Se encuentra embebida en la SparkSession

## SparkSession

- Desde Spark 2.0, es la capa unificada desde donde se puede acceder a las 3 APIs: RDD, DataFrame y Dataset. Es también una capa por encima del SparkContext

Nuevo punto de entrada desde la versión 2.0

Punto de entrada para leer/escribir datos

Trabaja con metadatos

Responsable de la configuración de Spark

Gestión de recursos del cluster

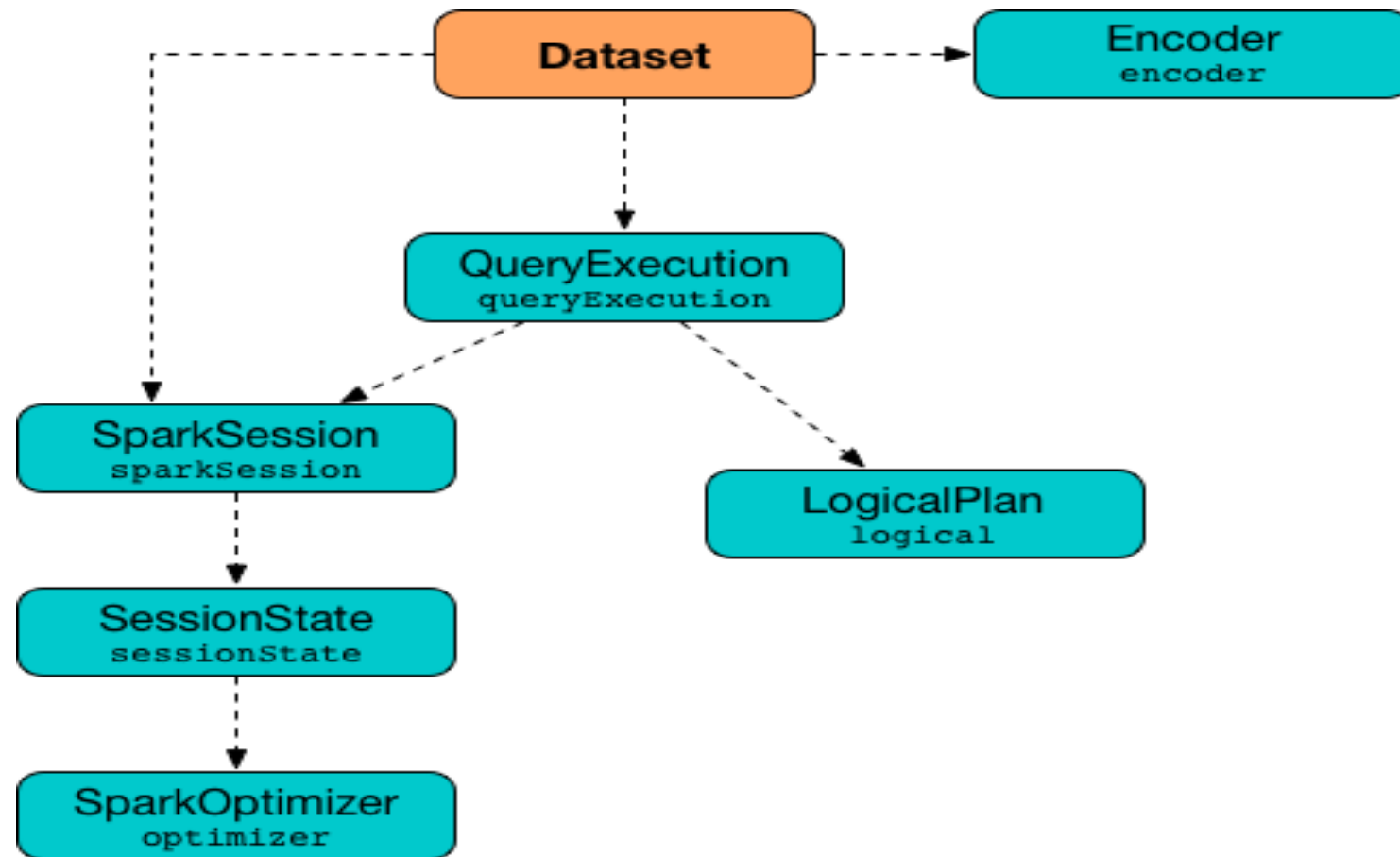
Genera Datasets/DataFrames

Encapsula:

SparkContext, SQLContext, StreamingContext y SparkConf

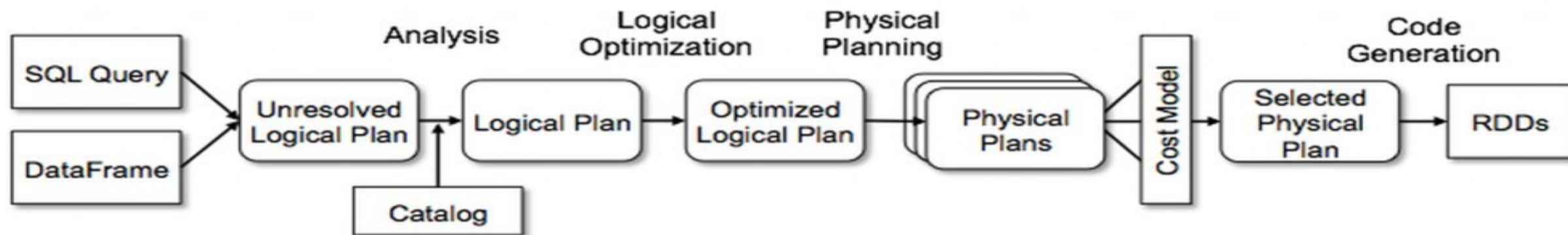
Una vez que se crea una instancia de SparkSession, puede configurar las propiedades de configuración SQL de tiempo de ejecución de Spark.

```
spark.conf.set("spark.sql.shuffle.partitions", 6)
```



```
xx val sparkSession =  
SparkSession.builder().appName("MySparkSession").master("local[*]").getOrCreate()  
  
xx val df = sparkSession.read.option("header",  
"true").csv("hdfs://namenode:9000/data/sales.csv")
```

## Catalyst





```
scala> spark.sql("EXPLAIN EXTENDED SELECT * FROM example3").show(500, false)
```

```
== Parsed Logical Plan ==
'Project [*]
+- 'UnresolvedRelation `example3`

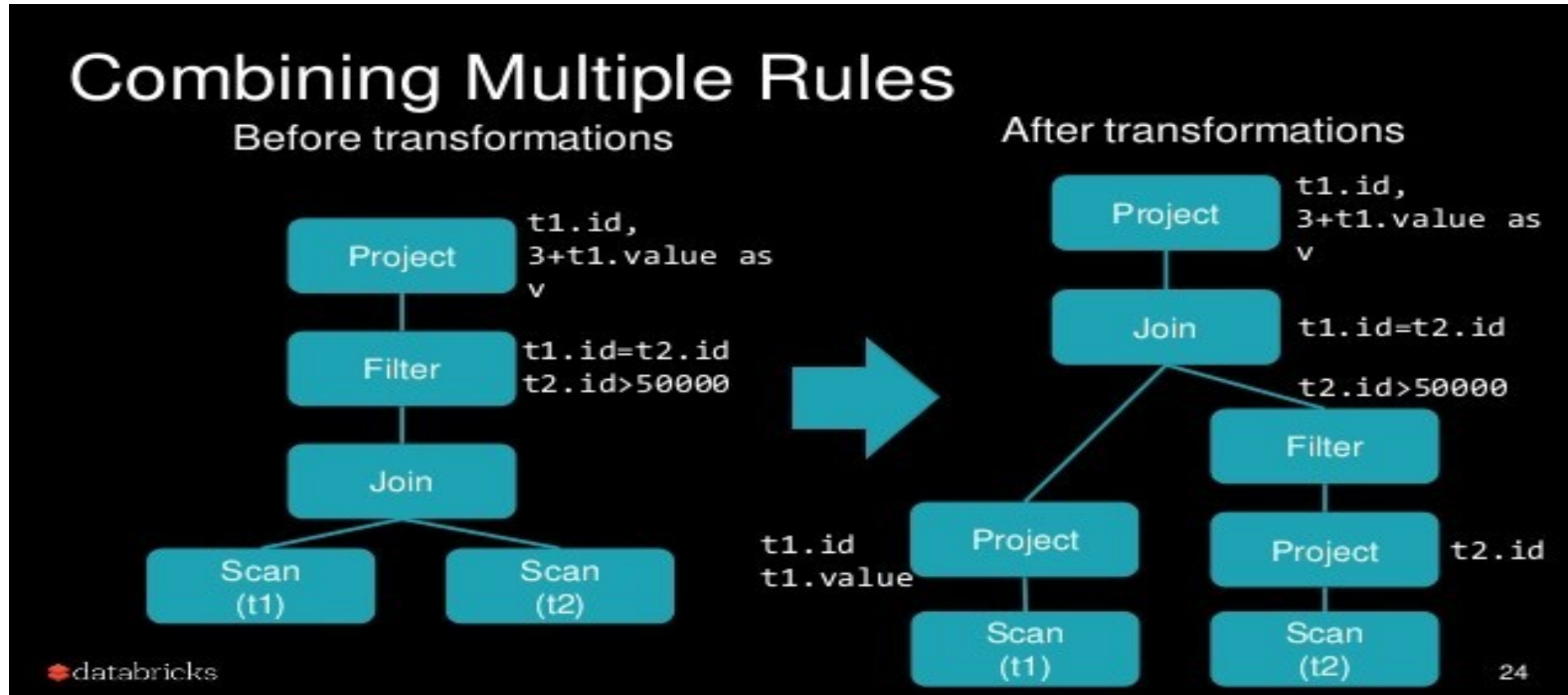
== Analyzed Logical Plan ==
id: int, name: string, age: int
Project [id#1255, name#1256, age#1257]
+- SubqueryAlias example3
   +- Relation[id#1255,name#1256,age#1257] parquet

== Optimized Logical Plan ==
Relation[id#1255,name#1256,age#1257] parquet

== Physical Plan ==
*FileScan parquet default.example3[id#1255,name#1256,age#1257] Batched: true, Format: Parquet, Location: InMemoryFileIndex[file:/tmp/example3], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<id:int,name:string,age:int>|
```

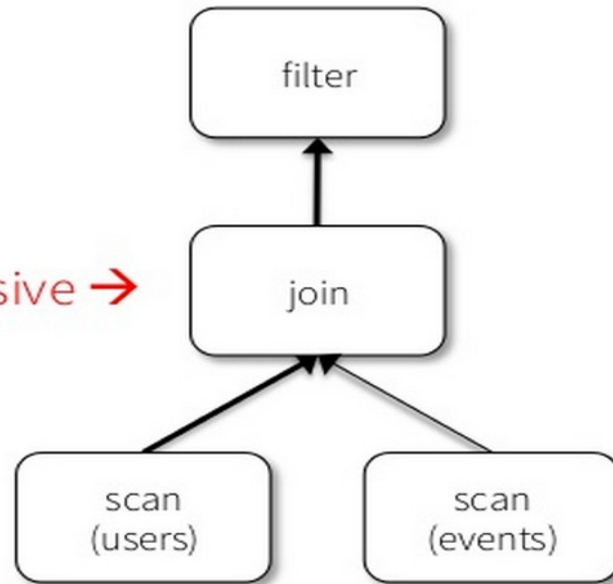
## Reglas

- Genéricas
- Específicas de cada datasource (Ejemplo: pushdown)



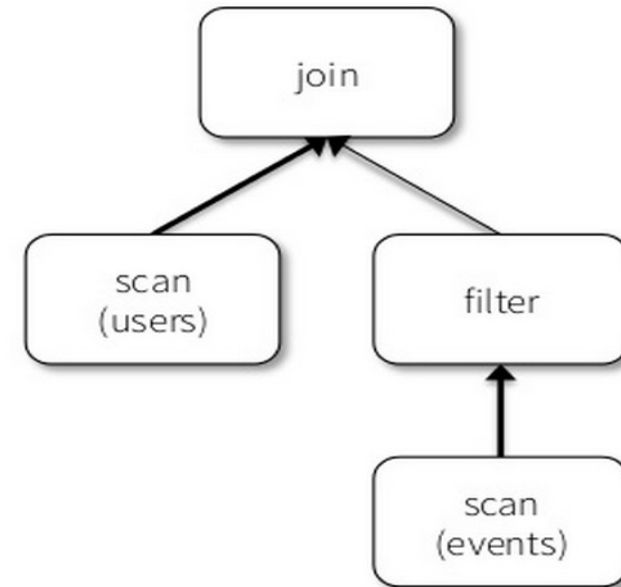
```
joined = users.join(events, users.id == events.uid)  
filtered = joined.filter(events.date >= "2015-01-01")
```

logical plan



this join is expensive →

physical plan



A través de `SparkSession`, se puede acceder a todas las funciones de Spark SQL como lo haría a través de `SQLContext`. En el ejemplo de código a continuación, creamos una tabla en la que emitimos consultas SQL.

```
xx zipDF.createOrReplaceTempView("zip_table")
xx zipDF.cache()

xx val resultsDF = spark.sql("SELECT city, pop, state, zip FROM
zip_table")

xx resultsDF.show(10)
```

```
xx import org.apache.spark.sql.Session
xx val spark = Session.builder().appName("Spark SQL basic
example").config("spark.some.config.option", "some-
value").getOrCreate()

xx import spark.implicits._
xx val df =
spark.read.json("examples/src/main/resources/employee.json")
df.show()

xx df.groupBy("age").count().show()

xx df.createOrReplaceTempView("employee")

xx val sqlDF = spark.sql("SELECT * FROM employee")

xx sqlDF.show()
```

<https://archive.apache.org/dist/spark/docs/3.3.3/sql-programming-guide.html>

```
xx case class Persona(nombre: String, edad: Int)

xx val personas = sc.textFile("examples/src/main/resources/
people.txt").map(_.split(",")).map(p => Persona(p(0),
p(1).trim.toInt)).toDF

xx people.registerAsTempTable("personas")

xx val adolescentes =
sql("SELECT nombre FROM personas WHERE edad >= 13 AND
edad<= 19")

xx val nameList = adolescentes => "Nombre: " + t(0)).collect()

xx val adolescentes =
people.where('edad>= 10').where('edad<= 19').select('nombre')
```

# 4. Conectores



# Spark SQL: Datasources

## Incluídos:

- CSV
- JSON
- Parquet
- Avro
- ORC
- Binary
- JDBC
  - Oracle
  - MySQL
  - Postgres

## Comunidad:

- [Cassandra](#)
- Mongo
- Elasticsearch
- Neo4j

## File Systems:

- HDFS
- Amazon S3
- Azure Storage
- Google Cloud Storage

## Data Sources supported by DataFrames



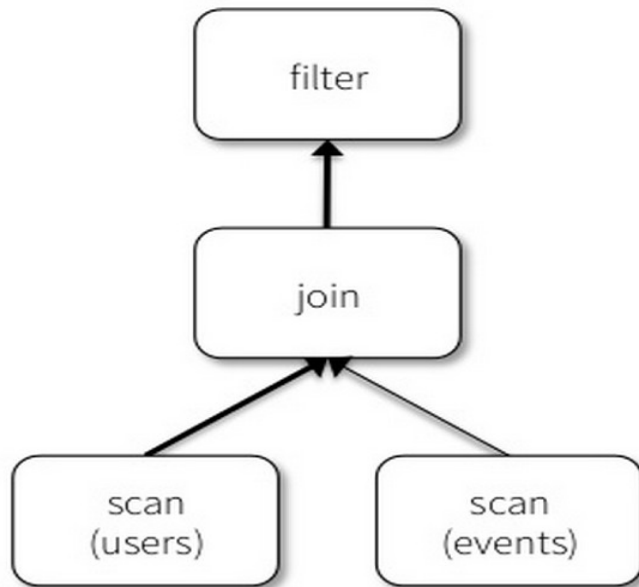
<https://archive.apache.org/dist/spark/docs/3.3.4/sql-data-sources-binaryFile.html>

<https://archive.apache.org/dist/spark/docs/3.3.4/cloud-integration.html>

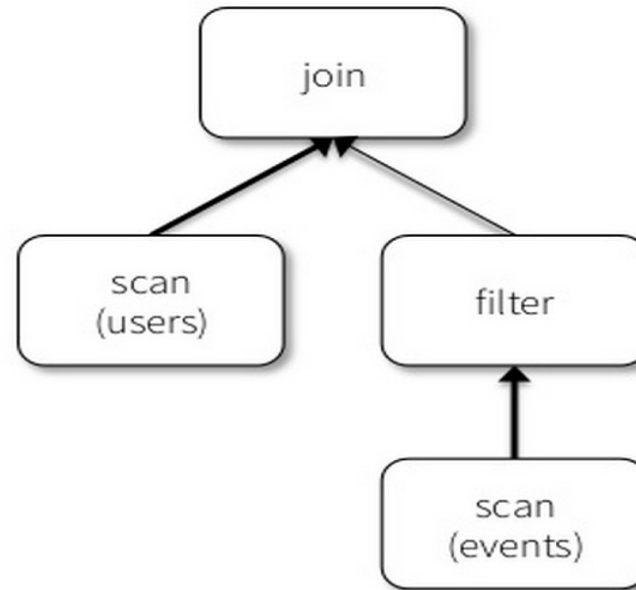
# Una capa única de acceso de datos

```
joined = users.join(events, users.id == events.uid)  
filtered = joined.filter(events.date > "2015-01-01")
```

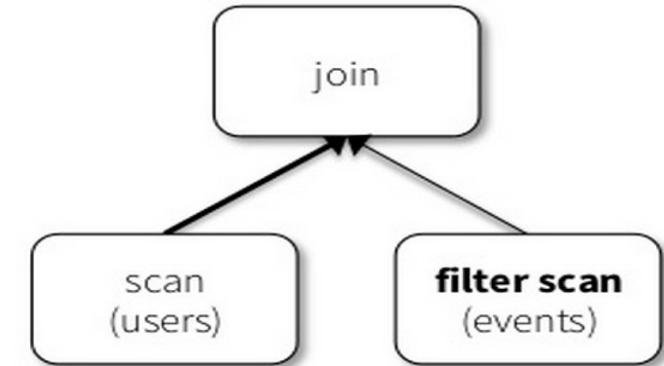
logical plan



optimized plan



optimized plan  
with intelligent data sources



```
// Reading a Cassandra table
spark.read
  .format("cassandra")
  .option("spark.cassandra.connection.host", "192.168.23.102")
  .option("keyspace", "mykeyspace")
  .option("table", "mytable")
  .load()

// Writing data into a Cassandra table
spark
  .createDataFrame(Seq((10000, -1, -1, -2)))
  .toDF("ttlCol", "k", "c", "v")
  .write
  .format("cassandra")
  .option("spark.cassandra.connection.host", "192.168.23.102")
  .option("keyspace", "mykeyspace")
  .option("table", "mytable")
  .option("ttl", "1000")
  .mode("APPEND")
  .save()
```

Modos de escritura  
para cualquier  
datasource:

- Append
- Overwrite
- ErrorIfExists
- Ignore

```
import org.apache.spark.sql._

val people = sc.textFile("s3n://bigdata-utad/data/people.txt")

val schemaString = "name age"

val schema = StructType(schemaString.split(" ").map(fieldName => StructField(fieldName, StringType, true)))xx

val rowRDD = people.map(_ .split(",")).map(p => Row(p(0),p(1).trim))

val peopleSchemaRDD = sqlContext.applySchema(rowRDD,schema)

peopleSchemaRDD.registerTempTable("people")

val results = sqlContext.sql("SELECT name FROM people WHERE age >= 13 AND age <= 19")

results.map(t => "Name: " + t(0)).collect().foreach(println)
```

```
./bin/spark-shell --conf  
"spark.mongodb.read.connection.uri=mongodb://127.0.0.1/test.myCollection?  
readPreference=primaryPreferred" \  
    --conf "spark.mongodb.write.connection.uri=mongodb://127.0.0.1/test.myCollection" \  
    --packages org.mongodb.spark:mongo-spark-connector_2.12:10.2.2  
val dataframe = spark.read  
    .format("mongodb")  
    .option("database", "people")  
    .option("collection", "contacts")  
    .load()
```

<https://www.mongodb.com/docs/spark-connector/current/batch-mode/batch-read-config/>

# Spark SQL: JSON

```
// Primitive types (Int, String, etc) and Product types (case classes) encoders are
// supported by importing this when creating a Dataset.
import spark.implicits._

// A JSON dataset is pointed to by path.
// The path can be either a single text file or a directory storing text files
val path = "examples/src/main/resources/people.json"
val peopleDF = spark.read.json(path)

// The inferred schema can be visualized using the printSchema() method
peopleDF.printSchema()
// root
// |-- age: long (nullable = true)
// |-- name: string (nullable = true)

// Creates a temporary view using the DataFrame
peopleDF.createOrReplaceTempView("people")

// SQL statements can be run by using the sql methods provided by spark
val teenagerNamesDF = spark.sql("SELECT name FROM people WHERE age BETWEEN 13 AND 19")
teenagerNamesDF.show()
// +-----+
// |  name|
// +-----+
// |Justin|
// +-----+

// Alternatively, a DataFrame can be created for a JSON dataset represented by
// a Dataset[String] storing one JSON object per string
val otherPeopleDataset = spark.createDataset(
  """"{"name":"Yin","address":{"city":"Columbus","state":"Ohio"}}"" :: Nil)
val otherPeople = spark.read.json(otherPeopleDataset)
otherPeople.show()
// +-----+-----+
// |      address|name|
// +-----+-----+
// |[Columbus,Ohio]| Yin|
// +-----+-----+
```



 Calle Playa de Liencres, 2 bis  
(entrada por calle Rozabella)  
Parque Europa Empresarial  
Edificio Madrid  
28290 Las Rozas, Madrid

 900 373 379  [info@u-tad.com](mailto:info@u-tad.com)

 [SOLICITA MÁS INFORMACIÓN](#)



CENTRO ADSCRITO A:

 **Universidad  
Camilo José Cela**

PROYECTO COFINANCIADO POR:

